ALGORITMOS Y ESTRUCTURAS DE DATOS

GRUPO N°:	**CURSO:** K1153

PROFESOR: Ing. Pablo Damián Mendez

DÍA DE CURSADA: Miércoles

EN EL TURNO: Noche

TRABAJO PRÁCTICO GRUPAL 2026

Sistema Offline de Drones Militares

INTEGRANTES	EMAIL Y USUARIO GITHUB	LEGAJO
Alexis Gonzalo Naufamer	anaufamer@frba.utn.edu.ar anaufamer	222.548-7
Magno Paúl Fernández Colmán	mfernandezcolman@frba.utn.edu.ar mfernandezcolman-wq	222.300-4

	FECHAS	FIRMA Y ACLARACIÓN DEL DOCENTE
ENTREGADO EL	04/11/2026	
CORREGIDO		
APROBADO		

OBSERVACIONES DEL PROFESOR:

1. Introducción y Descripción del Problema

El presente trabajo práctico tiene como objetivo desarrollar el sistema de guía de apoyo offline para drones militares, previniendo la inoperatividad en caso de bloqueos de señal GPS. Para esto, el programa gestiona una cuadrícula de 200x200 donde el dron ejecuta comandos secuenciales. El sistema permite cargar rutas desde archivos binarios, validarlas, modificarlas en memoria RAM o disco, y visualizar el reporte final en formato HTML.

2. Hipótesis y Decisiones de Diseño

Para resolver el problema, tomamos las siguientes decisiones técnicas basándonos en lo visto en clase:

- **Uso de Memoria Estática (static):** Como la matriz es de 200x200 (40.000 casilleros) y cada struct Orden ocupa varios bytes, declararla de forma normal dentro del main o en las funciones nos generaba un error de Stack Overflow (desbordamiento de pila). Decidimos usar static Orden memoriaDron[FILAS][COLS] para alojarla en el segmento de datos.
- **Archivos Binarios (.dat):** Usamos fread y fwrite con los modos "rb", "wb" y "rb+". El modo "rb+" nos permitió sobrescribir un registro puntual en la Operación 4 usando fseek para retroceder el cursor tras encontrar el dato.
- **Validación Estricta de Rutas:** Para cumplir con la regla de "sin huecos" y "celdas aledañas", creamos la función sonAledanas() que usa valor absoluto (abs()) para medir que la distancia en X y en Y no sea mayor a 1, impidiendo además que el dron apunte a la misma celda donde ya está parado.

3. Desafíos y Dificultades del Desarrollo

Durante la programación nos encontramos con varios problemas que fuimos resolviendo en equipo:

- **Lectura de textos con espacios:** Al principio usábamos cin >> nombreArchivo, lo cual cortaba la lectura si el archivo se llamaba "ataque final.dat" y nos rompía el menú. Lo solucionamos implementando cin.getline() y limpiando el buffer de entrada.
- **Validación de bucles infinitos:** Nos costó darnos cuenta de que un usuario podía crear una ruta que diera vueltas en círculo (ej: 0,0 -> 0,1 -> 0,0). Lo solucionamos agregando una matriz booleana auxiliar (visitado[200][200]) en la carga y creación del archivo, que "pinta" por dónde pasó el dron y aborta si vuelve a pisar el mismo lugar.
- **Casilleros Fantasma:** En las opciones de corregir registros, nos dimos cuenta de que podíamos editar una celda vacía (donde el dron no pasaba) y arruinar la matriz. Agregamos un control if(!memoriaDron[x][y].ocupado) para bloquearlo.

4. División de Tareas

El desarrollo del trabajo práctico se hizo en conjunto de principio a fin mediante llamadas por Google Meet. Se eligió esta modalidad de programar a la par para poder discutir y facilitar de que ambos entendiéramos el cien por ciento de la lógica del sistema paso a

paso. Aunque siempre pensamos el código mirando la misma pantalla, nos turnamos para liderar la escritura de los distintos módulos y optimizar los tiempos:

Alexis Gonzalo Naufamer: Lideró la construcción del menú principal, las funciones que evitan que el programa se rompa si el usuario tipea mal (leerEntero, leerRuta, sonAledanas), la lógica guiada para crear misiones nuevas (Opción 3), el guardado de datos (Opción 6) y el diseño del radar visual en HTML (Opción 7).

Magno Paúl Fernández Colmán: Encabezó las validaciones críticas al cargar los archivos (Opción 1), asegurando que el dron despegue bien, no deje huecos en la ruta y no caiga en bucles infinitos. Además, armó el reporte paginado por consola (Opción 2) y programó las funciones para corregir registros (Opciones 4 y 5), encargándose de manipular los archivos directo en el disco usando fseek.

5. Diagrama de Bloques (Subprogramas)

La arquitectura del programa consta de un Menú Principal iterativo que llama a subprogramas independientes según la opción elegida.

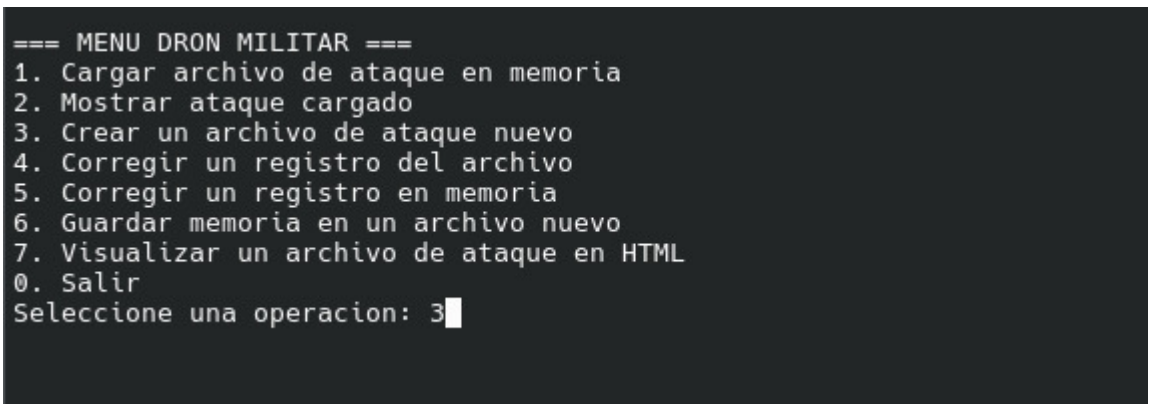
- **Principal:** main() controla el menú y almacena la matriz RAM.
- **Validaciones (Auxiliares):** leerEntero(), leerBool(), sonAledanas(), pedirSiguienteValido().
- **Módulo Archivos:** cargarArchivoEnMemoria() (Op 1), crearArchivoNuevo() (Op 3), guardarMemoriaEnArchivo() (Op 6).
- **Módulo Edición:** corregirRegistroArchivo() (Op 4), corregirRegistroMemoria() (Op 5)
- **Módulo Visualización:** mostrarAtaque() (Op 2), generarHTML() (Op 7).

Abrir el siguiente link “con Drawio” para visualizar el diagrama: https://drive.google.com/file/d/1vDNSSee9Hv9fLp27XyhNAVRAoDBp-ig9/view?usp=drive_link

6. Casos de Prueba (Capturas de Pantalla)

A continuación, mostramos el correcto funcionamiento del sistema:

Prueba 1: Menú principal, creación de un archivo válido y carga en memoria (Opción 3 y Opción 1)



```
Ruta y nombre del archivo a crear (ej: ataque.dat): ataque.dat

--- REGISTRO #1 ---
[!] Primer registro: Se establece DESPEGUE automaticamente (Sin ataques).
Coordenada inicial X (0 a 199): 0
Coordenada inicial Y (0 a 199): 0
Espera (0 o mas): 0
Siguiente X (0 a 199): 0
Siguiente Y (0 a 199): 1

--- REGISTRO #2 ---
[!] Posicion actual del dron fijada en: (0, 1)
Ataque Kamikaze  (1=Si, 0=No): 0
Aterrizaje       (1=Si, 0=No): 0
Espera (0 o mas): 0
Soltar Granada 1 (1=Si, 0=No): 1
Soltar Granada 2 (1=Si, 0=No): 0
Siguiente X (0 a 199): 0
Siguiente Y (0 a 199): 2
[!] El dron avanza hacia (0, 2).

--- REGISTRO #3 ---
[!] Posicion actual del dron fijada en: (0, 2)
Ataque Kamikaze  (1=Si, 0=No): 1
Espera (0 o mas): 0
Soltar Granada 1 (1=Si, 0=No): 0
Soltar Granada 2 (1=Si, 0=No): 0
[!] Mision finalizada (Aterrizaje/Kamikaze registrado). Cerrando archivo...
Exito: Archivo creado con 3 registros perfectamente conectados.
```

```
=== MENU DRON MILITAR ===
1. Cargar archivo de ataque en memoria
2. Mostrar ataque cargado
3. Crear un archivo de ataque nuevo
4. Corregir un registro del archivo
5. Corregir un registro en memoria
6. Guardar memoria en un archivo nuevo
7. Visualizar un archivo de ataque en HTML
0. Salir
Seleccione una operacion: 1
Ingrese ruta y nombre del archivo a cargar: ataque.dat
Exito: Archivo validado y cargado en memoria correctamente (3 pasos).
```

Prueba 2: Mostrar ataque cargado (Opción 2)

```
=== MENU DRON MILITAR ===
1. Cargar archivo de ataque en memoria
2. Mostrar ataque cargado
3. Crear un archivo de ataque nuevo
4. Corregir un registro del archivo
5. Corregir un registro en memoria
6. Guardar memoria en un archivo nuevo
7. Visualizar un archivo de ataque en HTML
0. Salir
Seleccione una operacion: 2

=== REPORTE DE ATAQUE EN MEMORIA (Pag. 1) ===

[Paso #1] Coordenada (0, 0):
  -> 1. INSTRUCCION: Despegue
  -> 7. MOVER A: (0, 1)

[Paso #2] Coordenada (0, 1):
  -> 3. ACCION: Soltar Granada 1
  -> 7. MOVER A: (0, 2)

[Paso #3] Coordenada (0, 2):
  -> 5. ACCION: Ataque Kamikaze

=== FIN DEL RECORRIDO ===
```

Prueba 3: Corrección de un registro del archivo. (Opción 4)

```
=== MENU DRON MILITAR ===
1. Cargar archivo de ataque en memoria
2. Mostrar ataque cargado
3. Crear un archivo de ataque nuevo
4. Corregir un registro del archivo
5. Corregir un registro en memoria
6. Guardar memoria en un archivo nuevo
7. Visualizar un archivo de ataque en HTML
0. Salir
Seleccione una operacion: 4
Ruta y nombre del archivo a corregir: ataque.dat
Coordenada X a buscar (0-199): 0
Coordenada Y a buscar (0-199): 1

--- DATOS ENCONTRADOS EN (0, 1) ---
Espera: 0 | Granada1: 1 | Granada2: 0 | Kamikaze: 0 | Aterrizaje: 0 | Despegue: 0 | Siguiente: (0,2)

--- INGRESE NUEVOS DATOS ---
[!] Nota: Despegue, Aterrizaje y Kamikaze estan protegidos para no romper la estructura.
Nueva espera (0 o mas): 2
Soltar Granada 1 (1=Si, 0=No): 0
Soltar Granada 2 (1=Si, 0=No): 0
Siguiente X (0 a 199): 0
Siguiente Y (0 a 199): 2
Exito: Registro actualizado en el disco duro.
AVISO: Si desea ver este cambio reflejado en el sistema, vuelva a cargar el archivo (Opcion 1).
```

Prueba 4: Corrección de un registro en memoria (Opción 5) y Guardar memoria en un archivo nuevo (Opción 6).

```
=== MENU DRON MILITAR ===
1. Cargar archivo de ataque en memoria
2. Mostrar ataque cargado
3. Crear un archivo de ataque nuevo
4. Corregir un registro del archivo
5. Corregir un registro en memoria
6. Guardar memoria en un archivo nuevo
7. Visualizar un archivo de ataque en HTML
0. Salir
Seleccione una operacion: 5
Coordenada X a modificar (0-199): 0
Coordenada Y a modificar (0-199): 1

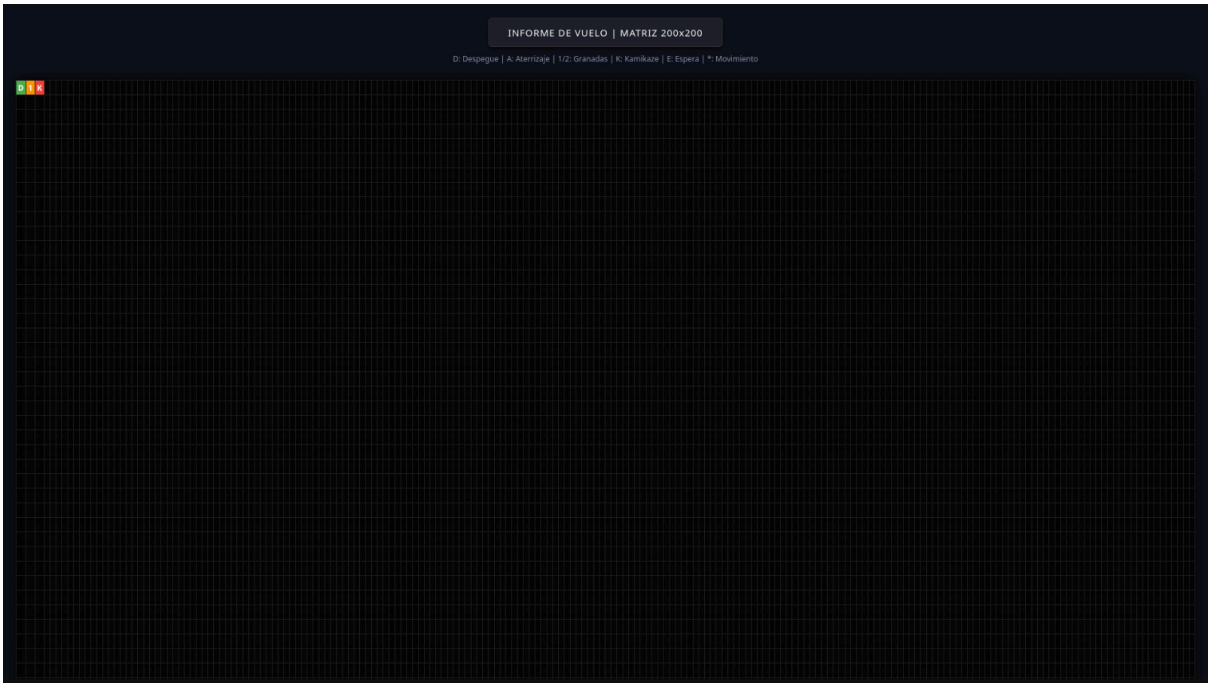
--- DATOS ACTUALES EN RAM (0, 1) ---
Espera: 2 | Granada1: 0 | Granada2: 0 | Kamikaze: 0 | Aterrizaje: 0 | Despegue: 0 | Siguiente: (0,2)

--- INGRESE NUEVOS DATOS ---
[!] Nota: Despegue, Aterrizaje y Kamikaze estan protegidos.
Nueva espera (0 o mas): 0
Soltar Granada 1 (1=Si, 0=No): 1
Soltar Granada 2 (1=Si, 0=No): 0
Siguiente X (0 a 199): 0
Siguiente Y (0 a 199): 2
Exito: Registro actualizado en memoria RAM.
```

```
=== MENU DRON MILITAR ===
1. Cargar archivo de ataque en memoria
2. Mostrar ataque cargado
3. Crear un archivo de ataque nuevo
4. Corregir un registro del archivo
5. Corregir un registro en memoria
6. Guardar memoria en un archivo nuevo
7. Visualizar un archivo de ataque en HTML
0. Salir
Seleccione una operacion: 6
Ruta y nombre del archivo binario a crear (ej: backup.dat): ataque.dat
Exito: Se guardaron 3 registros en 'ataque.dat'.
```

Prueba 5: Resultado del HTML generado (Opción 7)

```
=== MENU DRON MILITAR ===
1. Cargar archivo de ataque en memoria
2. Mostrar ataque cargado
3. Crear un archivo de ataque nuevo
4. Corregir un registro del archivo
5. Corregir un registro en memoria
6. Guardar memoria en un archivo nuevo
7. Visualizar un archivo de ataque en HTML
0. Salir
Seleccione una operacion: 7
Nombre del archivo HTML a generar (ej: ataque.html): ataque.html
Exito: Archivo HTML 'ataque.html' generado. Abrelo en tu navegador.
```



7. Declaración de uso de Herramientas de IA

- **Nombre de la herramienta:** Google Gemini.
- **Dirección URL:** https://gemini.google.com/
- **Motivo de uso:** Se utilizó a modo de "consulta" para detectar el porqué de un bucle infinito que sufríamos al leer archivos binarios, y para comprender cómo funcionaba la inyección de código CSS básico dentro de un archivo generado en C++ usando fprintf.
- **Respuesta otorgada y Adaptaciones:** La IA nos sugirió formas de limpiar el búfer del teclado cuando fallaba el cin >>. Adaptamos el código integrando cin.ignore() en funciones personalizadas nuestras (leerEntero) para no ensuciar el main. También usamos su sugerencia visual para los colores del HTML (Opción 7), adaptando el tamaño de las celdas a píxeles fijos para que se viera prolijo.